Week 15: End to End Data System

Keeyong Han

Table Of Contents

1. Overall Picture
2. Databases (Data Warehouse/Lake and NoSQL)
3. Data Pipelines (Batch Processing & Streaming Processing)
4. Course Update
5. Guest Speaker
    a. Hyunjoon Jung at Adobe: GenAI and its productionization

Overall Picture

Data Flow

Site traffic &
3rd party data

1

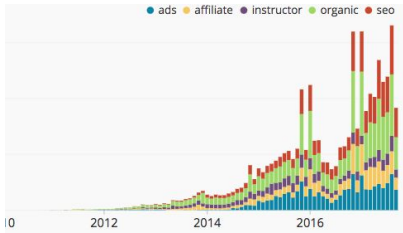
ELT

ETL

Data warehouse

Data Team

2



Business
Insights

3

Site Services

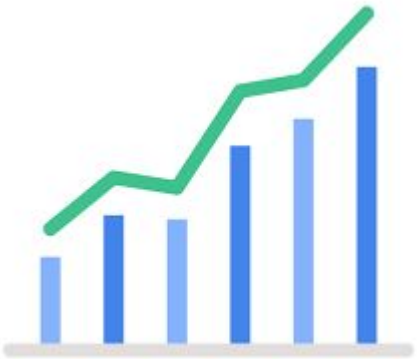
Scalable Product Improvement

Vision of Data Organization

Build **Leverage** for the company through **Trustworthy** Data

How to Add Values: 1. Decision Science

- Equips decision makers for more effective choices
  - **data informed** decisions
    - vs. data driven decisions



How to Add Values: 2. Product Science

- Scalable “product” improvement by algorithm (ML/AI)
  - Ex) Personalized recommendation, search and so on

Skillsets of Data Engineers (**Software** Engineer)

Troubleshooting
Data Literacy
PII Protection (GDPR, CPRA)
Use GenAI tools everyday

SQL, Data Warehouse
Python, Java (github)

ETL (Airflow)
Cloud (AWS,GCP,Azure)

Big Data Processing (Spark/Hadoop)
Streaming Data Processing (Kafka)

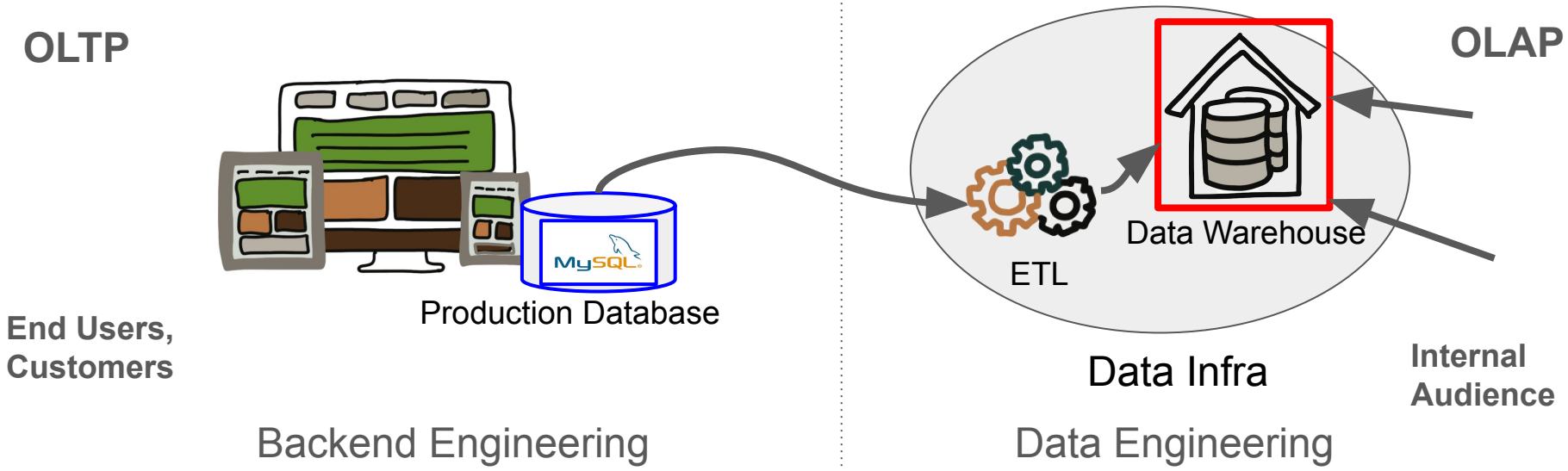
Container Technology (**Docker**,K8S)

NoSQL (MongoDB, ElasticSearch, Cassandra, DynamoDB …)

Databases
(Data Warehouse/Lake and NoSQL)

Data Warehouse: A separate database only for Data Stuffs

- Building a data warehouse is the first step to becoming a true data org
- Data Warehouse should be separate from the production database.
  - Dedicated database for leveraging data vs. production database

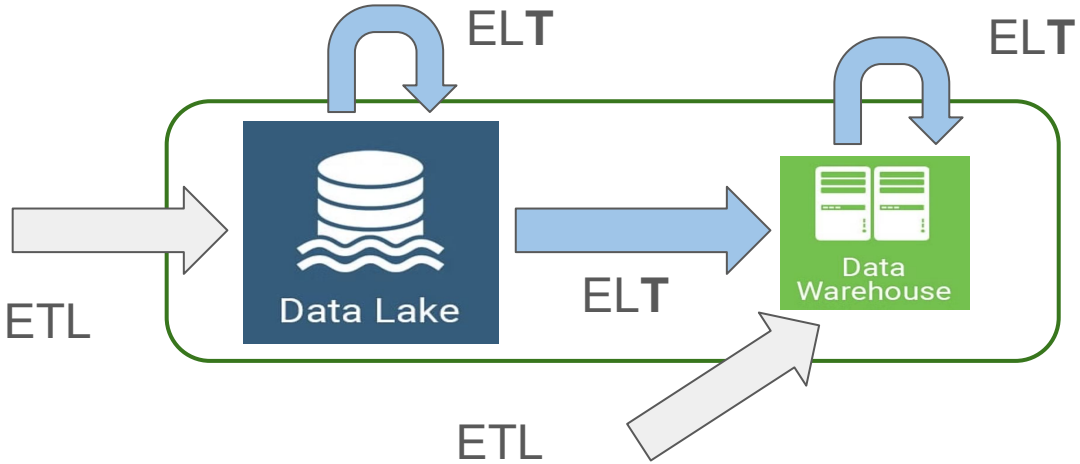


Difference between OLTP and OLAP (both are databases)

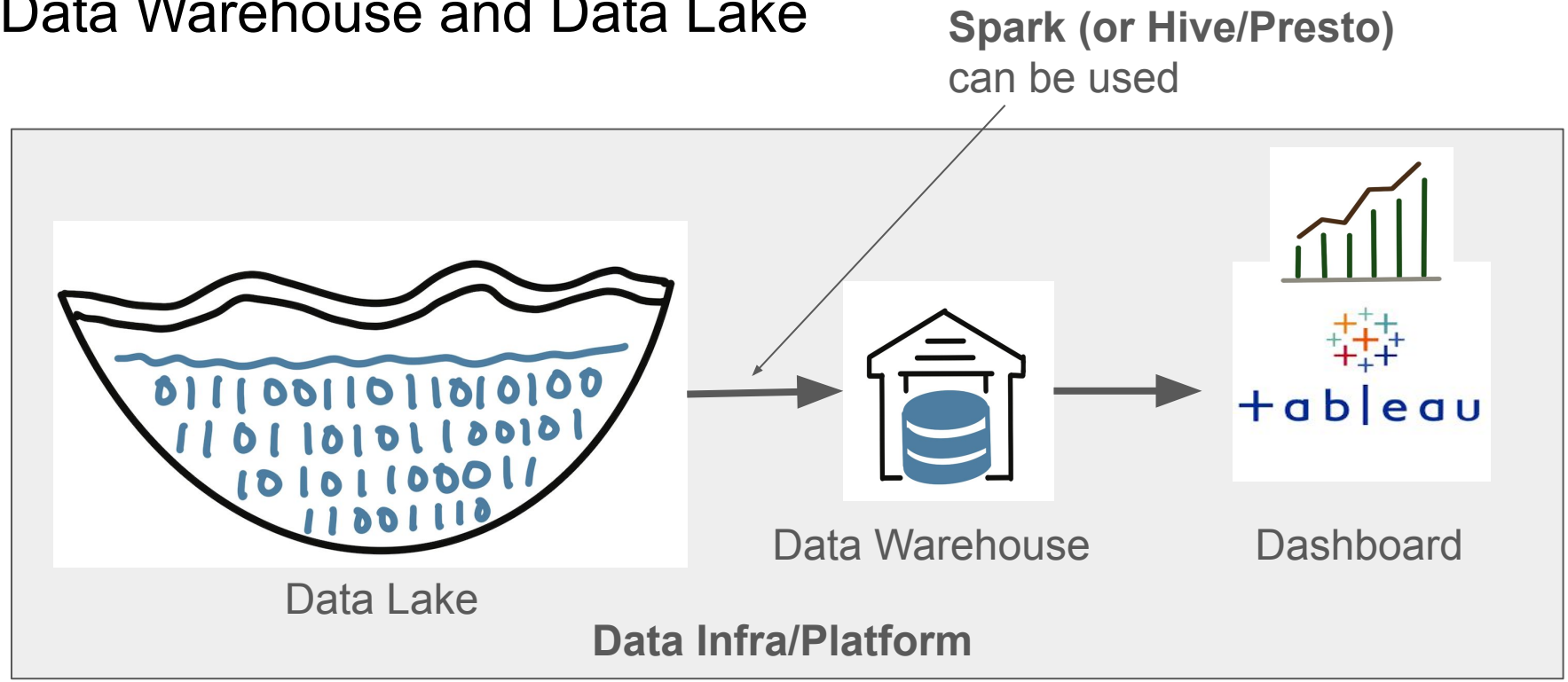
- OLTP: Production Database
  - Used to store data needed for operating the service/product
    - processing speed is more important
  - Managed by **Backend** engineering
  - MySQL, PostgreSQL, Oracle, …
    - A single server solution with limited capacity (Scale Up)
- OLAP: Data Warehouse
  - Used to store data needed for data analysis and other data related work
    - size of data to handle is more critical
    - In other words, scalability is the key (and cost-effectiveness)
  - Managed by **Data** engineering
  - Snowflake, BigQuery, Redshift, Hive/Presto, ClickHouse, …
    - A group of servers behind the scene (Scale Up and Out)

Data Lake

- Order of magnitude bigger than Data Warehouse
  - Can store both **structured** data and **unstructured** data in cost effective fashion
  - Data can be stored in raw form without retention policy (as long as there are no legal issues)
- More like a cloud storage
  - AWS S3, GCP GCS, …



Data Warehouse and Data Lake



LakeHouse: Data Lake + Data Warehouse (Delta Lake, **Iceberg**)

Evolution of Data Infra

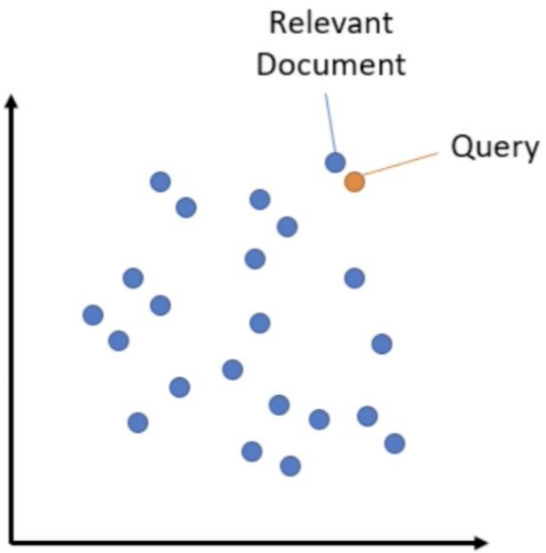
1. Data Warehouse
2. Data Lake + Data Warehouse
3. Data Lake
    a. From this point, usually separation of compute and storage is normal
    b. Very specific purpose data warehouses are introduced (called “Data Mart”)
4. Data LakeHouse

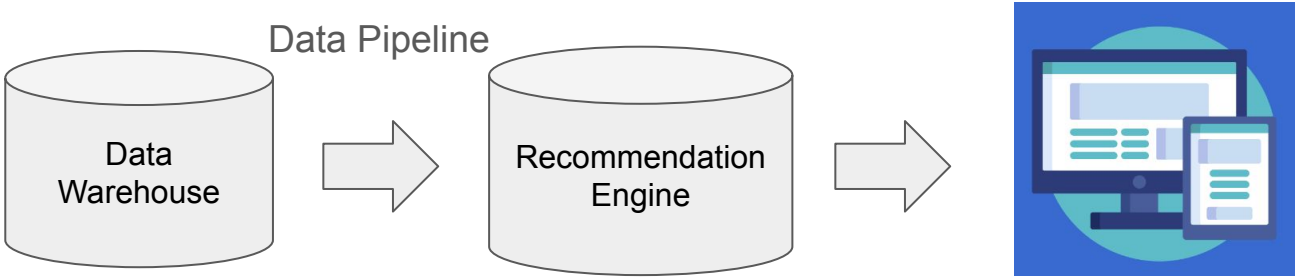
What is NoSQL

- High Volume, Low Latency and Flexible Schema
- Non-relational databases
- Schema-less or loose schema
  - Schema can change over time
- Key-Value Storage with different flavors
  - Key/Value Storage, Document Store, Wide Column Storage Search Engine
  - Example: Redis, MongoDB, Cassandra/CouchDB, ElasticSearch/VESPA
- Used in Production
  - Response time is critical (100ms)
  - Needs to be scalable (a few billion records)

What is Embedding and VectorDB

- Purpose of Embedding
  - Dimensionality Reduction
  - Similarity Measurement (“semantics”)
- VectorDB
  - Convert documents and a query to the same embedding space as vectors
  - Search can be done a similarity search (“Euclidian”, “Cosine”, …)

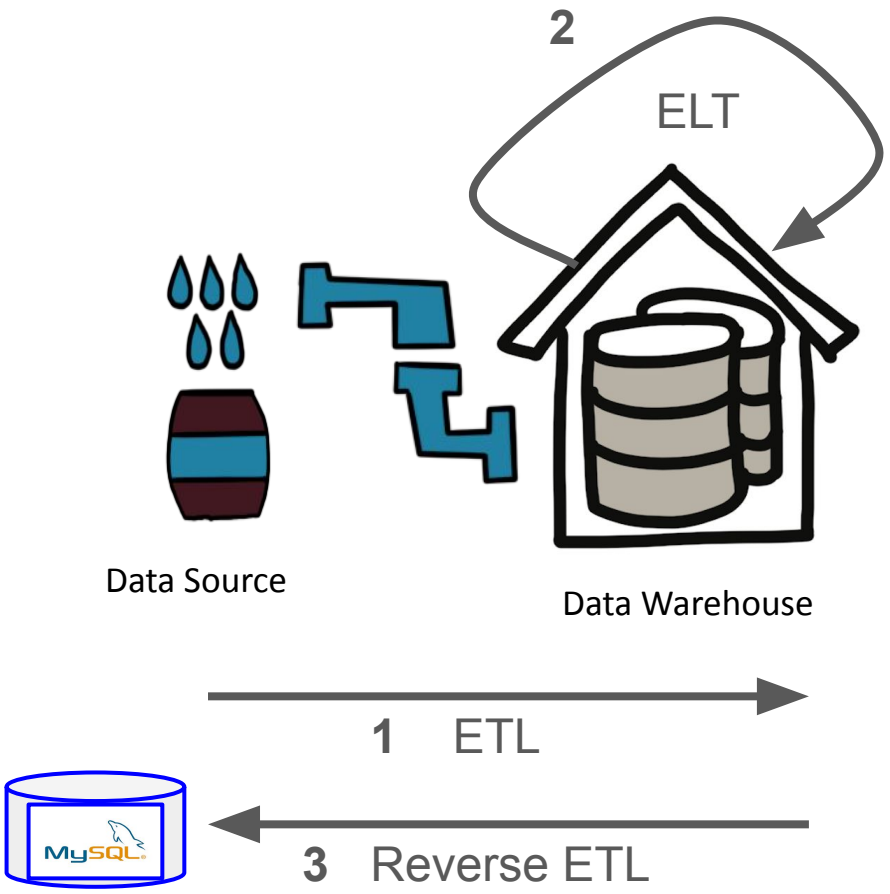




Data Pipelines

Types of Data Pipeline

1. ETL
2. ELT (“T”)
3. Reverse ETL



Key Concepts

- Full Refresh, Incremental Update
- Idempotency
  - SQL Transaction along with try/except
- Backfill
  - Rerunning a data pipeline to fill incomplete or missing data
  - This is where Airflow has strengths
- Upsert
  - Snowflake supports an operation called MERGE
  - Update existing records and insert new records
    - Most efficient way of ensuring the primary key uniqueness

What is Airflow?

- A de facto Data Pipeline management framework
  - You can write data pipelines and can manage them (scheduling, triggering, …)
  - Python modules are provided along with the Web UI
  - So popular that most Cloud providers supports Airflows as their service
    - Google Cloud supports Airflow (“Cloud Composer”)
- An open source started from Airbnb along with Superset
- Data pipeline in Airflow is called DAG (Direct Acyclic Graph)

Airflow Details

- Airflow Variables
- Airflow Connections
- Airflow presumes all pipelines are doing incremental update
- Key DAG parameters
  - schedule, catchup, start_date, …
- Context Variables
  - logical_date, ds, …

ETL vs. ELT (Both are Pipelines not databases)

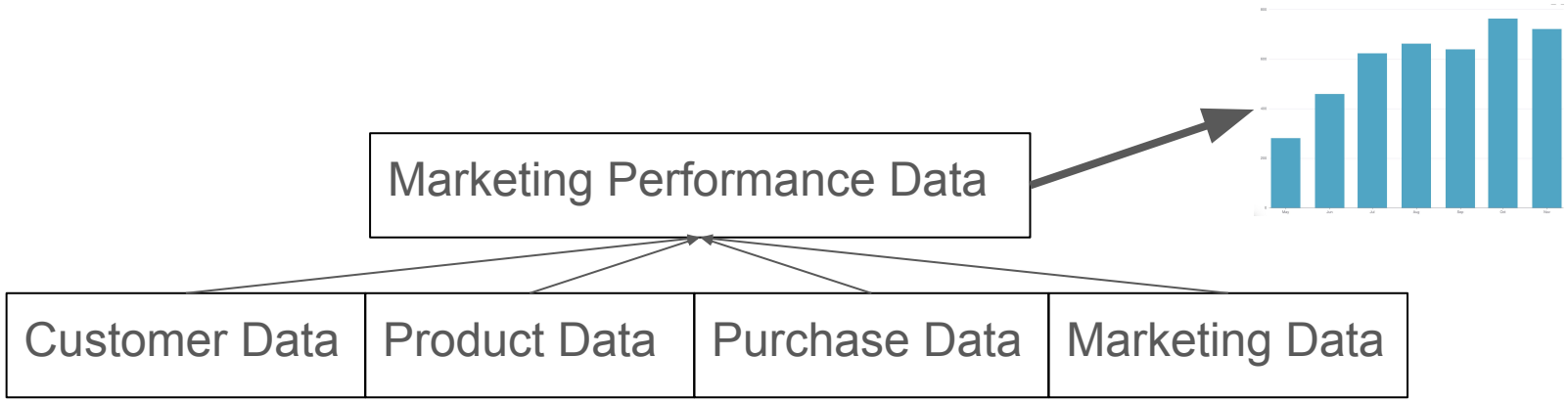
ETL ≠ OLTP & ELT ≠ OLAP

- ETL
  - The process of bringing data from outside the data system to inside
    - Data system can be data warehouse or data lake
  - Done by data engineers
  - One of the key data sources is the production database (OLTP)
- ELT
  - The process of manipulating data inside the data system to create new data (usually more abstracted and cleaned)
  - The most popular tool is dbt
  - Done by data analysts or analytics engineers
  - Input to ELT can be in the data lake

Airflow is used to schedule and/or develop pipelines

What is the future of ELT?

- We build ETLs so that we can get data insights in some ways
- We run ELTs to get the insights by creating abstract layers
- Data quality validation becomes extremely important



Increasing Importance of Data Quality

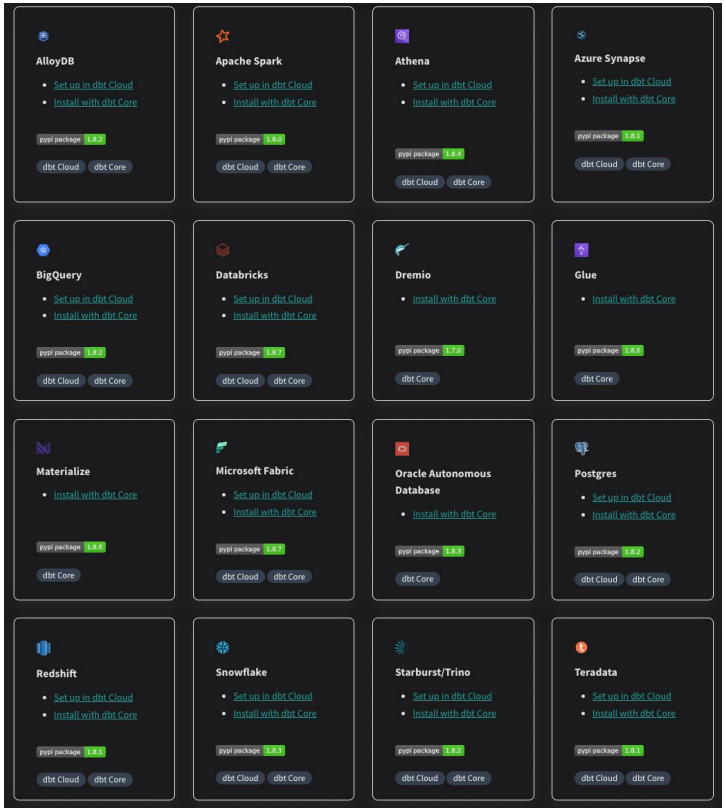
- Input/output checks (in particular for ELTs)
- More diverse quality inspections
- Lineage checks
- Understanding data change history

This is why **dbt** is needed

- Maintaining data quality is more important than ever
  - Reducing cost/effort and increasing productivity
  - Main reasons to have **data governance**

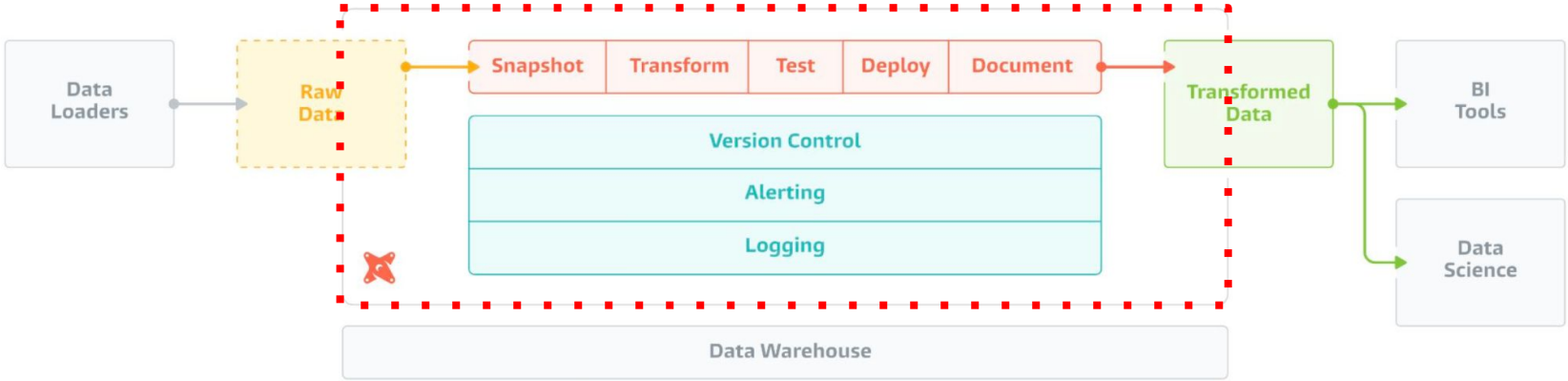
What is dbt?

- Data Build Tool (https://www.getdbt.com/)
  - ELT Open-source: In-warehouse data transformation
  - Commercialized by a company called dbt Labs
    - Valued at $4.2B once
  - Coined the term “Analytics Engineer”
- Supports various data warehouses
  - Redshift, **Snowflake**, BigQuery, Spark
- A cloud version is also available: dbt Cloud



dbt Components

- Data Models (models):
  - Manages tables in several tiers
  - CTAS (Create Table As Select) on steroids, lineage tracking
  - Includes Table, View, CTE, etc.
- Data Quality Validation (tests)
- Archives (snapshots)



When to Use dbt

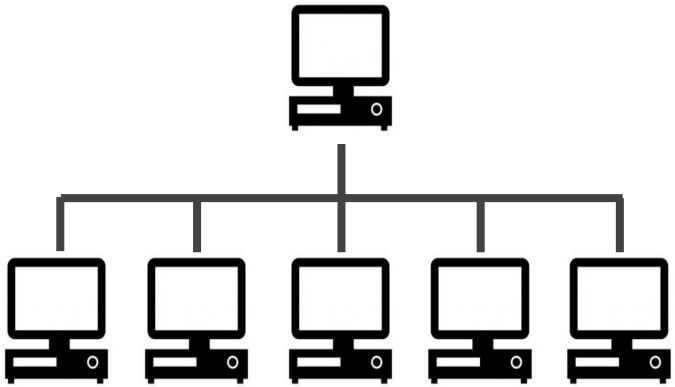
- Easy to understand data changes and enable rollbacks if necessary
- Data quality testing and error reporting (“tests”)
- Incremental updates for fact tables
- Change tracking for dimension tables (“history/archive” tables)
- Easy documentation generation
- Ability to track data lineage

Break

Big Data & Streaming Processing

Characteristics of Big Data Processing

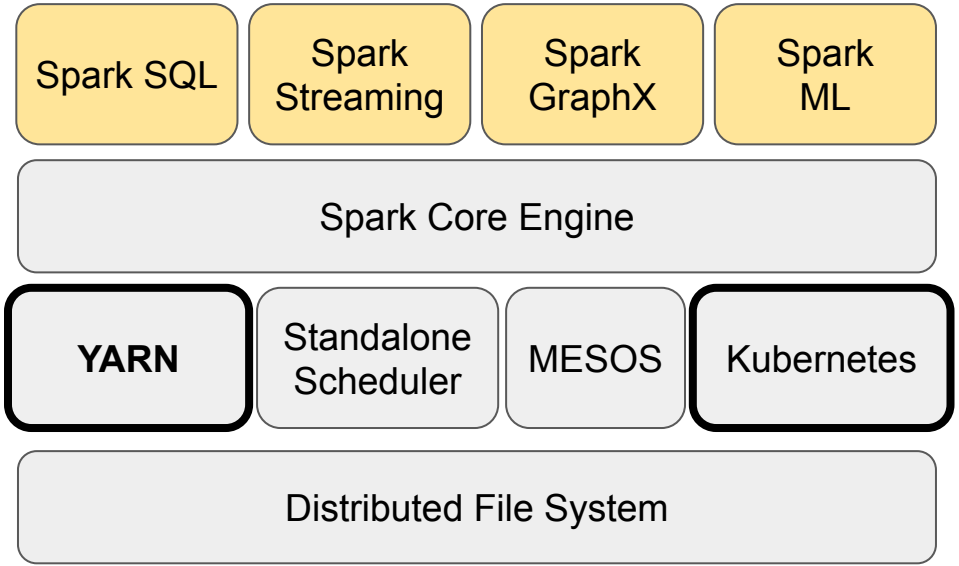
Challenges	Solutions
Need a way to store large volumes of data without loss	Distributed File System: HDFS, S3, Cloud Storage (Data Lake)
Sequential processing takes too long: Computing Power + Parallel Processing	Distributed Computing System MapReduce -> Hive/Presto -> Spark
Big data is often unstructured: SQL alone is not sufficient	DataFrame + SQL



Distributed System

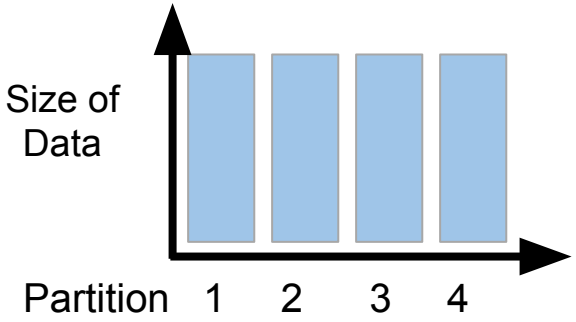
Spark System Overview

- An Apache open-source project in 2012 from UC Berkeley's AMPLab
  - Databricks
- Second-generation big data technology following Hadoop
- Provides diverse functionalities for big data processing!
  - DataFrame, SQL, ML, Streaming, Graph, …

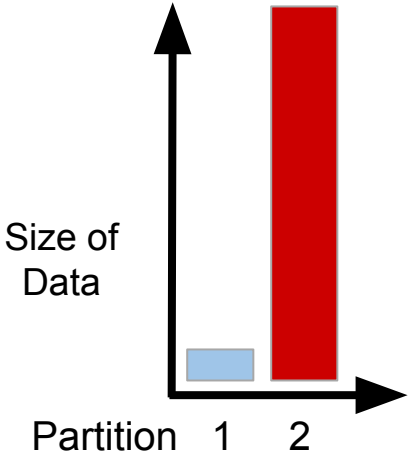


Key Concepts to Remember

- **Partitions**: data is split into multiple partitions (128MB or 256MB)
  - Each partition will be processed by a worker (CPU)
  - This is how parallelism is implemented
- When **shuffling** happens, new partitions will be created
  - GROUP BY or JOIN typically make shuffling happen
  - Some partitions can be too large -> **Data Skewness**

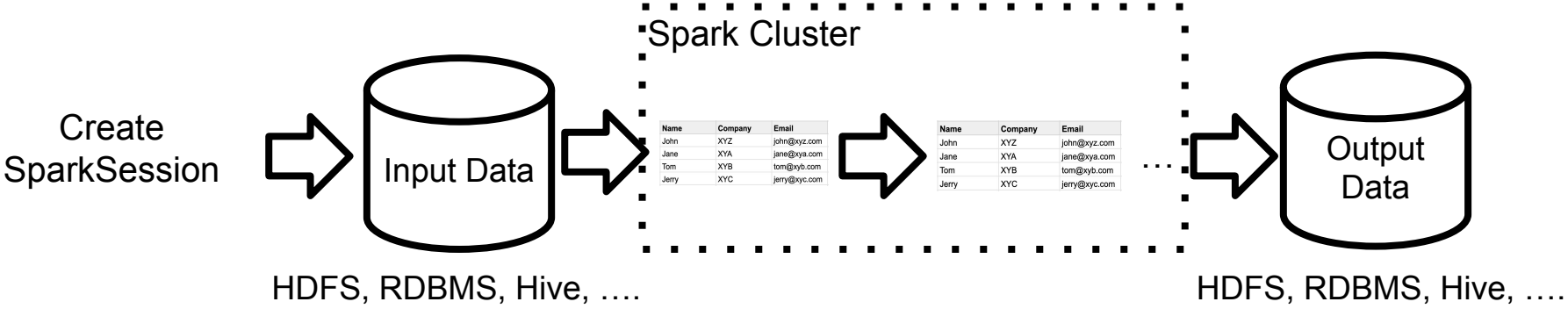


Shuffling



Overall Spark Program Flow

1. Creating a SparkSession
2. Loading the input data as a DataFrame
3. Performing data manipulation tasks (very similar to Pandas)
4. Continuously creating new DataFrames until the desired result is achieved
5. Saving the final result
6. Example code: [Pandas to Spark]

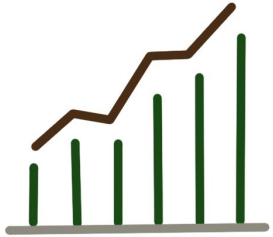


Data System Diagram

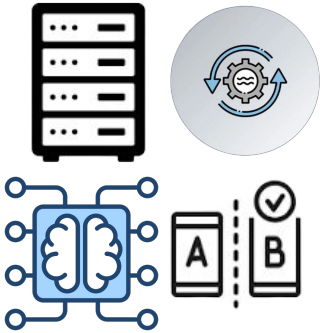
Data Sources

Data Pipelines

Data Storages



BI Tools



Data System: Advanced Data Processing Layer

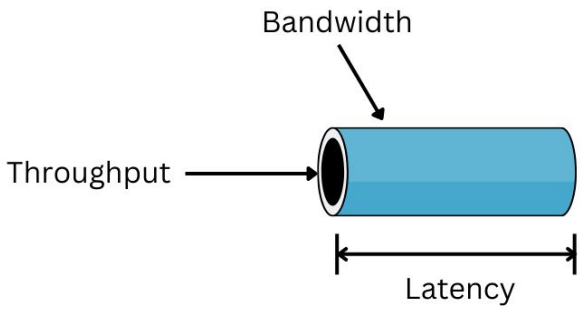
- Next Steps after foundation (data warehouse + data pipelines)
  - Small Data -> Big Data
  - Batch Processing -> Streaming Processing
  - ML models used in services
  - A/B testing becomes standardized
- The data team processes, models, and derives insights from data
  - ML Pipeline (training data collection, model building, model testing, model deployment, model monitoring)
- Key Tools:
  - Spark, NoSQL
  - Kafka/Kinesis
  - SageMaker, MLflow

Evolution of Data Processing

- Initially, Processing Begins with Batch
  - At this stage, the volume of data that can be processed is critical.
- The demand for real-time processing gradually increases over time
  - Real-time Processing vs. Semi-Real-time Processing
  - The need for shared data consumption grows: multiple data consumers emerge
  - Latency over throughput

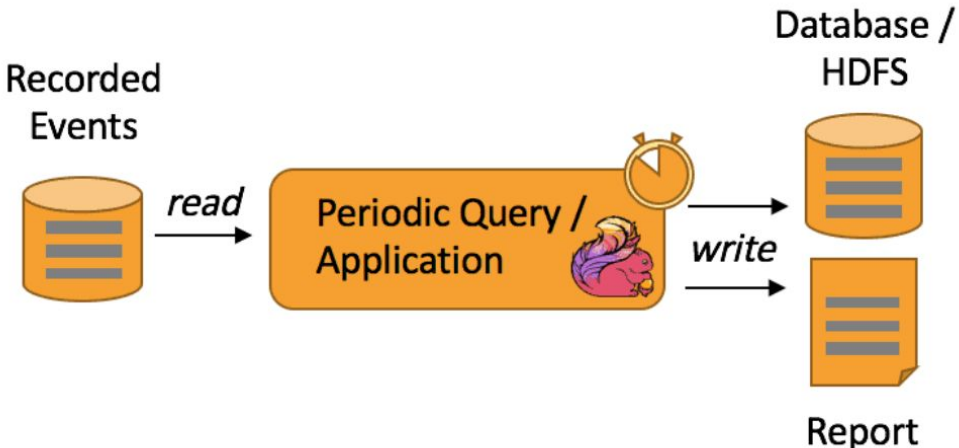
Throughput vs. Latency

- Throughput
  - The amount of data that can be processed within a given time unit (e.g., MB per second)
  - Higher throughput means a larger volume of data can be processed. This is especially important in batch systems (e.g., data warehouses).
- Latency
  - The time it takes to process data.
  - Lower latency means faster response times, which is crucial in real-time systems (e.g., production databases and NoSQL).
- Bandwidth = Throughput x Latency



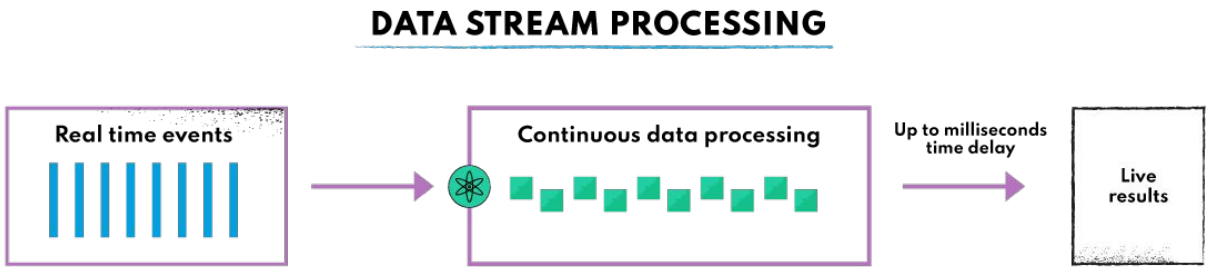
Batch Processing

- Periodically move or process data from one location to another
  - Throughput is critical
  - Processing intervals are typically in the range of minutes, hours, or days
- Data is collected and processed in batches
  - Pandas, Spark DataFrames, SparkSQL, …
  - Airflow is commonly used for scheduling tasks



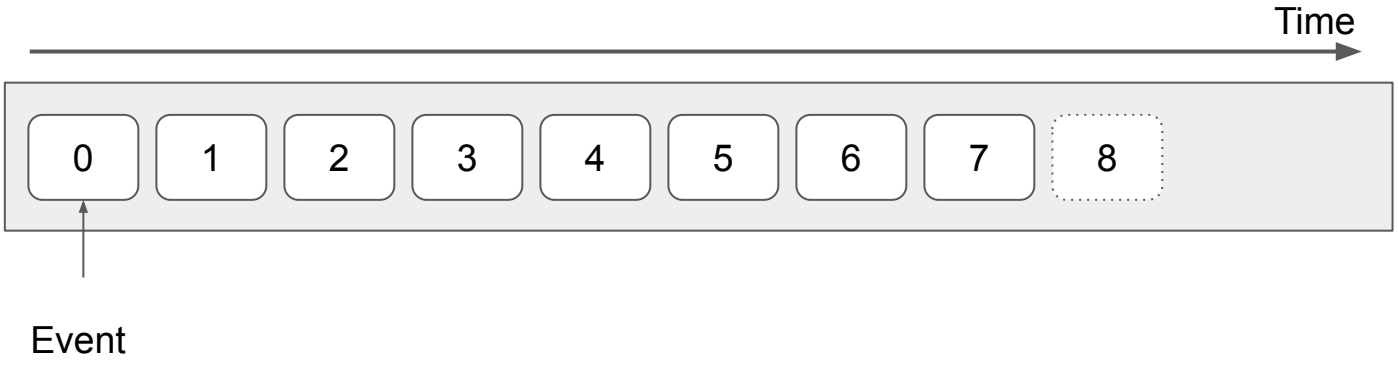
Streaming Data Processing

- Continuous Data Processing (vs. Batch Data Processing)
  - This type of data is called event or message at the individual record level
  - Some kind of a queue is needed to store events/messages
  - Producer and Consumer are needed
- Low Latency is very critical
  - Realtime vs. Semi-realtime (micro batch)



Streaming Data Processing (1)

- Next level of advancement after batch processing
  - The complexity of tasks like system management increases
- Continuous data processing at sub-second intervals
  - This data is typically called "events," which are characterized by being immutable
  - The ongoing flow of events is referred to as an "event stream" or “data stream”



Streaming Data Processing Structure

- A **Producer** (**Publisher**) generates data
- The generated data is stored in a system like a message queue
  a. Systems such as Kafka, Kinesis, and Pub/Sub are available
  b. Each data stream (called a topic in Kafka) can have its own data retention period
- **Consumers** (**Subscribers**) reads and processes data from the queue
  a. Each consumer maintains a separate pointer
  b. Multiple consumers can read data concurrently

Backpressure

- In streaming systems, data is typically generated at a steady rate
  - However, sometimes data generation can surge dramatically
  - Producer side
- Data needs to be processed in a timely manner at downstream stages
  - If the system cannot keep up with the incoming data rate, data accumulates, leading to delays, increased memory usage, and potential system failures—this is known as a **backpressure** issue
  - Consumer side
- One way to reduce backpressure is to introduce a queue in between.
  - While this approach can alleviate backpressure issues, it does not completely solve them.

Eventual Consistency?

- If we write a record to a distributed system with 100 servers, can we read that record immediately?
  - Will the record we just wrote be returned?
  - Typically, a single data block is stored across multiple servers (**Replication Factor**).
  - So, when data is written or modified, it takes time to propagate across the system.
  - Since reads usually target one of the multiple data copies, the data may or may not be available, depending on the propagation time.
- Strong Consistency vs. Eventual Consistency
  - Generally, if the system waits until replication is complete before returning, this is Strong Consistency.
  - If it returns immediately without waiting for replication, this is Eventual Consistency.

Streaming Data Technologies

- Needs for different types of services arise
  - Message queues for storing event data: Kafka, Kinesis, Pub/Sub, etc.
  - Processing systems for handling event data: Spark Streaming, Samza, Flink, etc.
  - Analytics/Dashboards for this type of data analysis: Druid

Cloud Pub/Sub

amazon KINESIS

Course Update

Group Project Presentation

- Group Project Presentation Duration: 10 mins each (including 2 mins QA)
- Presentation Order
  - 7, 3, 10, 4, 11, 8, 5, 12, 6, 9
- Presentation Sequence
  - Problem Statement
  - Historical/Realtime Dataset
  - System Architecture
  - DB Schema
  - ETL/ELT process (Airflow, dbt)
  - Visualization **(must be live demo!)**
  - Conclusion (summary, lessons, future work and so on)
- Three things to submit
  - Presentation Report: 10PM 05/08 due
  - Final Report: noon 05/12 due
  - Peer Review: 11PM 05/12 due

Here's a fair random shuffle of the numbers 3 through 12:

7, 3, 10, 4, 11, 8, 5, 12, 6, 9

Final Exams

- Will cover from Airflow to Streaming Data + SQL
- The format will be the same as the mid-term
  - Part A (multi-choice questions): 40 questions (80 pts)
  - Part B (short-answer and long-answer questions): 10 questions (65 pts)
    - This will include 3 SQL questions
      - Study Easy “Advanced Select and Joins” & “Basic Aggregate Functions” at https://leetcode.com/studyplan/top-sql-50/
    - This will also include 3 short Python questions (fill in the blank types)
      - Make sure you go through Airflow codes (transaction, backfill, …)
- Schedule
  - 3:15-5:15 PM Wednesday, May 14 (BBC 202)

Final Exams: SQL questions

- https://leetcode.com/studyplan/top-sql-50/

Advanced Select and Joins		
The Number of Employees Which Report to Each Employee	Solution	Easy
Primary Department for Each Employee	Solution	Easy
Triangle Judgement	Solution	Easy

Basic Aggregate Functions		
Not Boring Movies	Solution	Easy
Average Selling Price		Easy
Project Employees I	Solution	Easy
Percentage of Users Attended a Contest	Solution	Easy
Queries Quality and Percentage		Easy

Guest Speaker

Hyunjoon Jung at Adobe

GenAI and its productionization